

6-June

Arrays

Arrays is a collection of datatypes either Primitives or non-primitive or objects & even complex datatypes, we can store in an array.

Syntax :

`datatype[] variable-name = new datatype[size];`

Store 5 roll numbers

`int [] rnos = new int [5];`

↓
reference
variable

↓
objects

`int [] rnos;`
↓

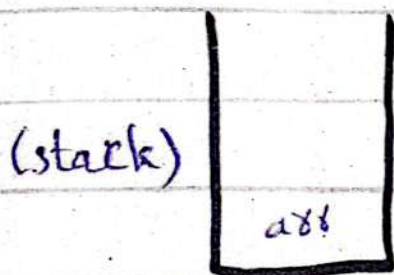
declaration of array. rnos is getting defined in the stack.

`arr = new int [5]; // initialization`

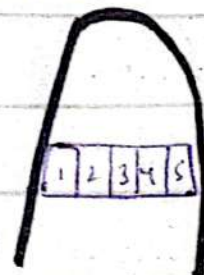
↓
actually here object is being created in the memory
called heap.
compile time memory allocated → Run Time memory allocated

`int [] arr = new int [5];` (Dynamic memory allocation)
↓ ↓
datatype reference variable
 ↓
 creating the object in the heap memory

JVM handles them.



↓
reference variable

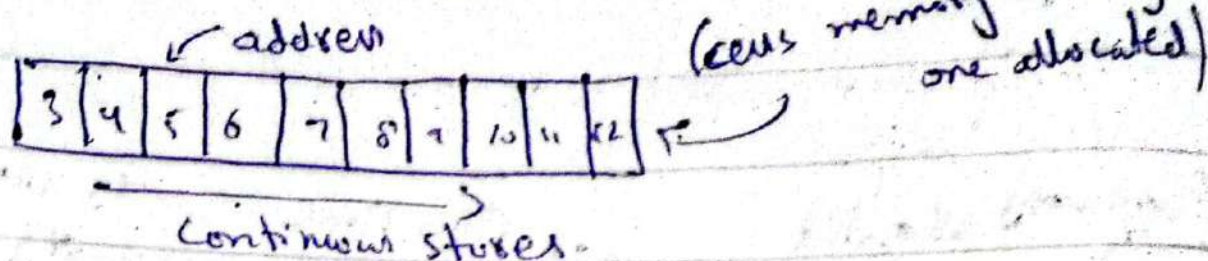


(heap) memory

object

Array in C++ stores continuous (Continuous memory allocation).

Why in C++ continuous



But in Java, it totally depends on JVM. Because there is no concept of Pointers in Java.

- ① Arrays object in heap.
- ② Heap objects are not continuous.
- ③ Dynamic memory allocated

Hence,

Array objects in Java may be ^{not} continuous -
even at google array definition is its a continuous data but in Java, internally may not be continuous.
↳ it depends on JVM. b/c JVM controls stack & heap memory.

index: 0 1 2 3 4 5

arr →

3	8	19	24 ⁹⁹	36	45
---	---	----	-----------------------------	----	----

Print (arr[0]) => Prints 0

Print (arr[2]) => Prints 19

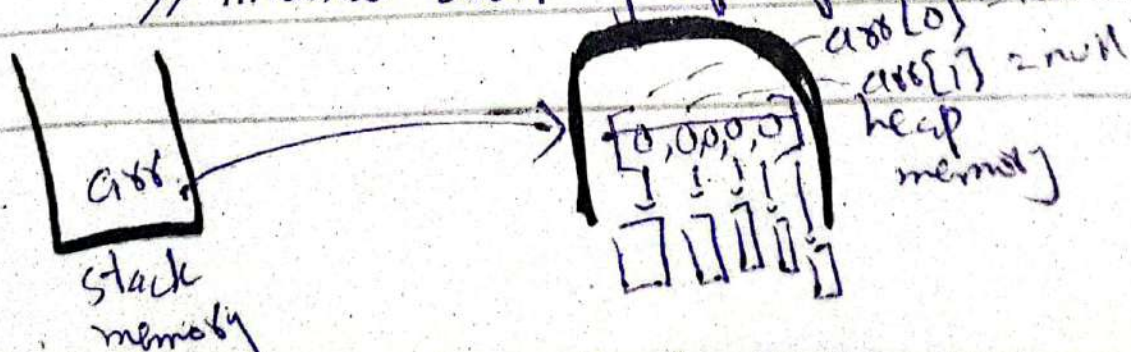
arr[3] = 99

new keyword is used to create an object

Primitives are store in stack memory but all other objects String, array etc all stores in heap memory.

String [] arr = new String [5];

// internal working of object



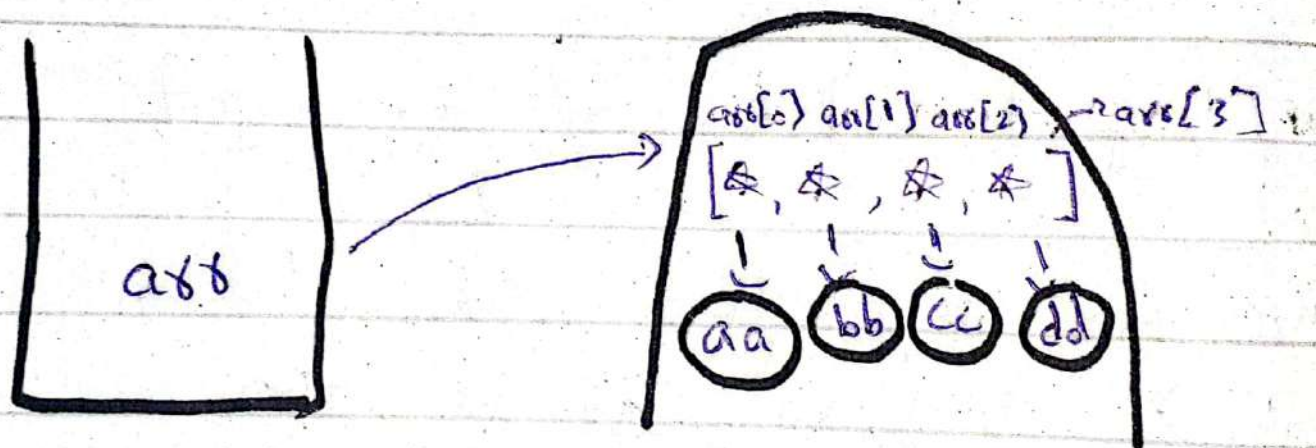
→ each particular element in array, it will be located in some other memory

For Example:

Input of 4 strings:

aa, bb, cc, dd

what's happen internally:



Mutability in Arrays:

```
Psvm {  
    int[] nums = {2, 3, 4, 5};  
    S.o.u.t (Arrays.toString(nums));  
    change(nums);  
    S.o.u.t (Arrays.toString(nums));  
}
```

```
static void change(int[] arr){  
    arr[0] = 99;  
}
```

nums → [2, 3, 4, 5]
arr → [2, 3, 4, 5] will arr[0] change from 2 to 99?

1st
output: [2, 3, 4, 5]
2nd
output: [99, 3, 4, 5]

b/c only copy / value pass of array
to change method not the
original one.

Ans: Yes it will change

This is called mutability in arrays.
b/c both of sep. variable pointed
to the same object [2, 3, 4, 5].

But remember:

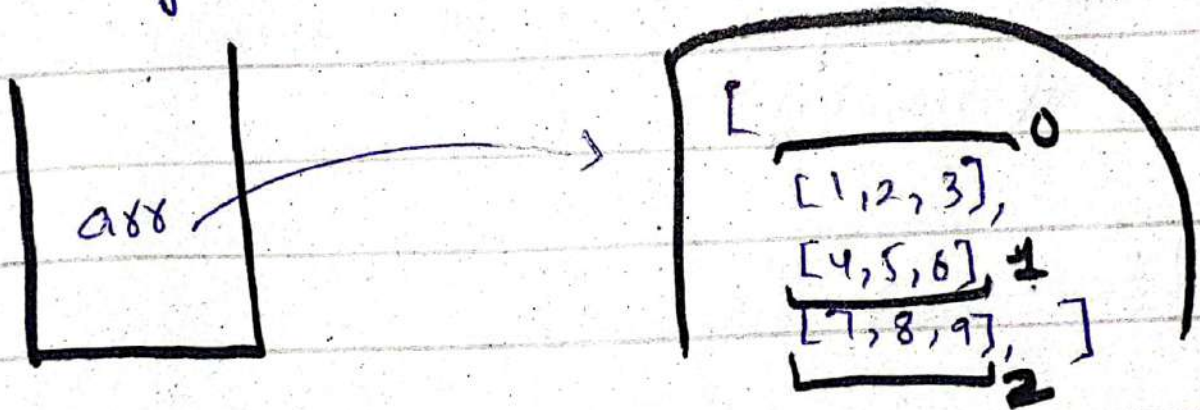
String are Immutable in Java
Arrays are Mutable in Java

Mutable :: mean you can change the
object.

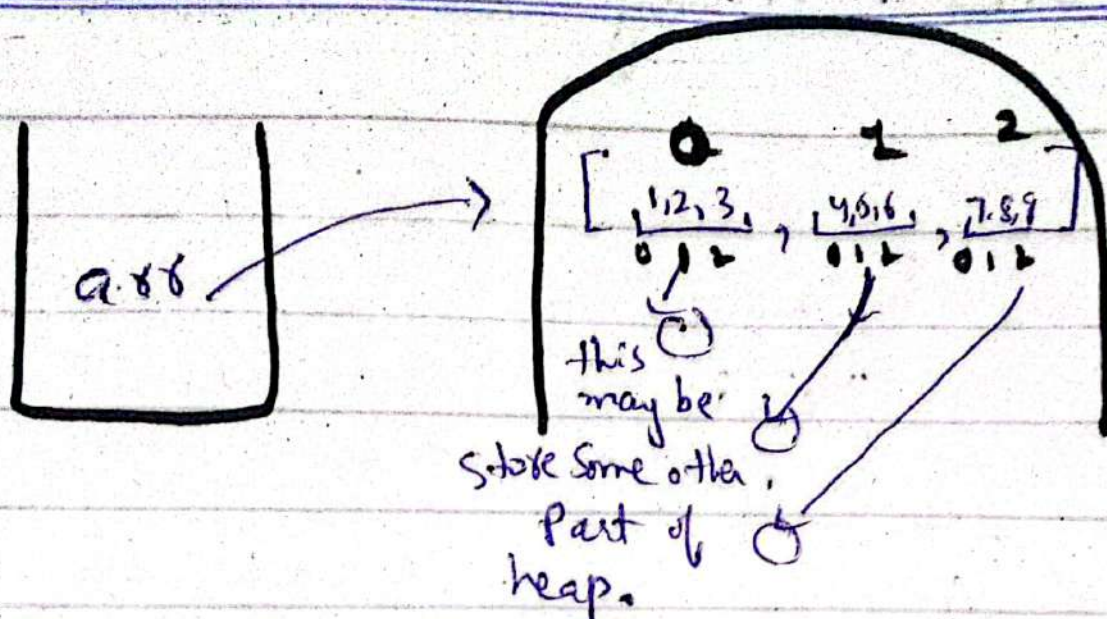
2D Array

```
int [3][3] arr = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9},  
};
```

Internally,



Imagine this is an
array of arrays.



↳ depends on JVM.

All may be store other parts of memory.

arr[1] → [4, 5, 6]

arr[1][0] → 4

Row Column

Columns

	0	1
0	23	18
1	9	27
2	39	8

3 Rows

2 in each 3 rows

there are 2 columns.

for (row = 0; row < 3; row++) { iterating row

// now ^{we} take every row

for (col = 0; col < 2; col++) { iterating column inside each row

arr[row][col] = input

}

}

Dry Run:

Row	Column	Take Input
0	0 1 2x	→
1	0 1 2x	→
2	0 1 2x	→

[1, 2, 3]	0th index
[4, 5]	1st index
[6, 7, 8]	2nd index

```

for (row = 0; row < arr.length; row++) {
    for (col = 0; col < arr[row].length; col++) {
        arr[row][col] = input;
    }
}

```


ArrayList:-

Why:- In array we know that what will be the size of our array but what if we don't know the size of array?

So, we use ArrayList.

↓
Part of collection Framework.

Syntax:-

ArrayList<Integer> list = new ArrayList<Integer>();

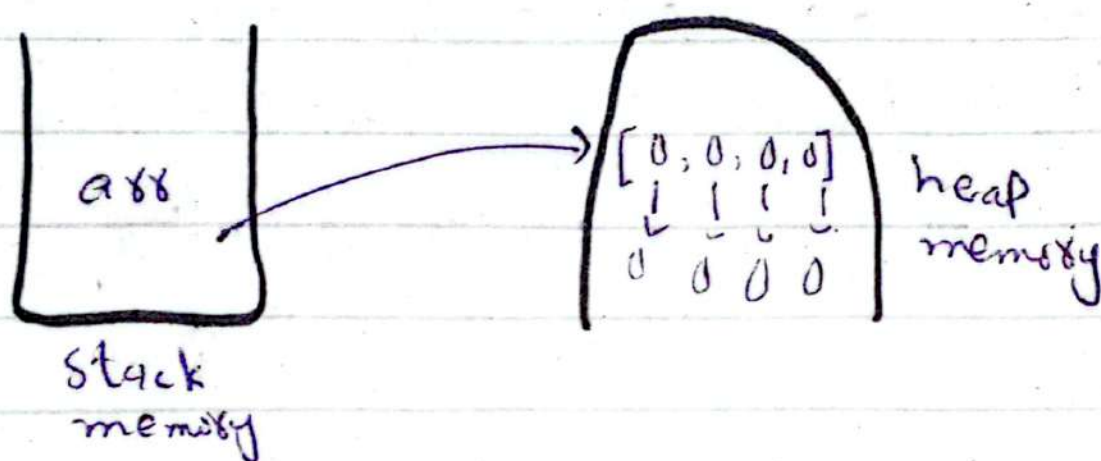
Annotations:
- wrapper class not primitive datatype
- initial capacity as well
- & can exceed to that as well
- here you can give constructors
- datatype (optional here)

list.add(23);

Annotations:
- datatype
- variable-name
- in this way add many you want to.

ArrayList is a class.

Internally, it works same as array like.



then why we say that size is not fixed?

1) Actually, size of an ArrayList is fixed internally.

2) Let's say, ArrayList fills by some amount then?

⇒ it will create a new ArrayList for example

⇒ may be double of the 1st or old ArrayList

⇒ then all the old ArrayList members / elements

⇒ copied to new one (double one) & old

elements gets **deleted!**

Ex:

[1, 2, 9, , ,] 1st

⇓

[1, 2, 9, 18, 20, ,] then

Maximum in Array.

I,

↓

↓

↓

↓

↓

1, 3, 23, 9, 18

max

↓

~~1~~ ~~3~~ 23